

Universidad Nacional de Costa Rica

Sección Regional Huetar Norte y Caribe

Diseño y Programación de Plataformas Móviles

I Ciclo 2026

NativeScript: Plataforma de Desarrollo Móvil Multiplataforma

Docente: Mag. Rachel Bolívar Morales

Curso: Diseño y Programación de Plataformas Móviles

Problemática asignada: Sistema de Gestión de Órdenes y Entregas

06 de junio de 2026

Estudiantes:

Noelia Fallas

Andrea Morera

Andrés Carranza

Jeremy Romero

Dorian Calderón

Resumen

El presente informe es una investigación técnica sobre NativeScript, un framework de código abierto para el desarrollo de aplicaciones móviles nativas multiplataforma. Se aborda su historia desde su creación por Telerik en 2014 hasta su ingreso a la OpenJS Foundation en 2020, la arquitectura interna basada en runtimes de JavaScript que permiten el acceso directo a las APIs nativas de iOS y Android, los lenguajes y frameworks que soporta, casos reales de uso en la industria y una comparación técnica con el desarrollo nativo en Android. El informe concluye con un análisis crítico de la plataforma en el contexto de la problemática asignada al grupo: el diseño de un sistema de gestión de órdenes y entregas.

Palabras clave: NativeScript, desarrollo móvil multiplataforma, framework JavaScript, iOS, Android, TypeScript.

Historia y Origen del Framework

Surgimiento y Primeros Años (2014–2016)

NativeScript nació como una iniciativa interna de Telerik, una empresa de software de origen búlgaro adquirida posteriormente por Progress Software Corporation. El proyecto fue anunciado públicamente en junio de 2014 con el objetivo de permitir a los desarrolladores web construir aplicaciones móviles verdaderamente nativas —sin recurrir a WebViews— utilizando únicamente JavaScript, CSS y una capa de marcado XML para la interfaz de usuario (NativeScript, 2014).

La versión 1.0.0 estable se publicó en mayo de 2015, apenas dos meses después del lanzamiento de la primera vista previa pública. Esta versión estableció las bases del framework: acceso directo a las APIs de iOS y Android desde JavaScript y renderizado nativo de la interfaz de usuario. Al respecto, el equipo de NativeScript (2014) afirmó que el framework

enables developers to use pure JavaScript language to build native mobile applications running on all major mobile platforms - Apple iOS, Google Android and Windows Universal. The application's UI stack is built on the native platform rendering and layout engine, using native UI components and because of that no compromises with the User Experience of the applications are done. (NativeScript, 2014, párr. 2)

En 2016, NativeScript 2.0 introdujo integración oficial con Angular 2, lo que permitió a los equipos de desarrollo web reutilizar directamente componentes y servicios construidos para la web en aplicaciones móviles nativas. Según InfoQ (2020), este hito amplió considerablemente la base de desarrolladores potenciales del framework al alinear NativeScript con el creciente ecosistema de Angular, permitiendo que equipos con experiencia web adoptaran el desarrollo móvil nativo sin cambiar de lenguaje ni de paradigma.

Expansión del Ecosistema (2017–2019)

En junio de 2017 la comunidad anunció soporte para Vue.js a través del plugin NativeScript-Vue, lo que permitió a los desarrolladores de ese ecosistema utilizar su framework preferido para construir aplicaciones móviles nativas. Este movimiento posicionó a NativeScript como una plataforma agnóstica en cuanto al framework de frontend, lo que lo distinguió de competidores como React Native, que en ese momento solo soportaba React. Durante este período el repositorio alcanzó más de 24.700 estrellas en GitHub y la comunidad contribuyó con más de 600 plugins disponibles en el marketplace oficial (NativeScript GitHub, 2023).

Transición a Gobernanza Abierta (2020–Actualidad)

A finales de 2019, Progress Software inició un proceso formal para transferir la responsabilidad del proyecto a la comunidad. En mayo de 2020, la empresa consultora nStudio asumió oficialmente el mantenimiento del framework. NativeScript (2020) describió este cambio como el resultado de "increased interest and improved engagement with the core framework" (párr. 2) impulsado por la comunidad de desarrolladores.

En diciembre de 2020, NativeScript ingresó a la OpenJS Foundation como proyecto en incubación. La OpenJS Foundation (2020) señaló que con este movimiento el proyecto fue colocado bajo el mismo paraguas de la Linux Foundation que alberga a Node.js, jQuery y webpack, garantizando así su neutralidad y sostenibilidad a largo plazo. Este cambio estructural desacoplaba la evolución del framework de los intereses de una única empresa privada (InfoQ, 2020).

Desde entonces, el framework ha continuado su desarrollo con versiones como NativeScript 8.0 en 2021, que introdujo soporte para Apple M1 e integración con Webpack 5. A partir de 2024, NativeScript también soporta plataformas como visionOS, Android TV y watchOS (NativeScript Blog, 2024).

Arquitectura y Funcionamiento

Visión General

La arquitectura de NativeScript se distingue de los frameworks híbridos como Ionic o Cordova en un aspecto fundamental: en lugar de renderizar la interfaz dentro de una WebView del navegador, NativeScript crea y manipula directamente los componentes nativos del sistema operativo. Según la documentación oficial del runtime de iOS (NativeScript, s. f.-a), "The iOS Runtime may be thought of as 'The Bridge' between the JavaScript and the iOS world" (párr. 3). Esto significa que un botón en una aplicación NativeScript es, en realidad, un UIButton de iOS o un android.widget.Button de Android.

El framework se organiza en cuatro capas principales que trabajan de manera coordinada:

- Capa de aplicación: el código JavaScript o TypeScript escrito por el desarrollador, organizado con Angular, Vue.js, React, Svelte o TypeScript puro.
- Capa de runtime: el motor que ejecuta el código JavaScript y gestiona la comunicación con las APIs nativas de cada plataforma.
- Capa de metadatos y bindings: un sistema que genera automáticamente, en tiempo de compilación, los tipos y referencias necesarios para llamar a cualquier API nativa desde JavaScript.
- Capa nativa: los propios SDKs de iOS (Objective-C/Swift) y Android (Java/Kotlin), a los que NativeScript accede sin intermediarios adicionales.

El Runtime como Núcleo del Framework

El runtime es el componente más crítico de NativeScript. Para Android, el runtime se construye sobre el motor V8 de Google; históricamente el runtime de iOS utilizaba JavaScriptCore, pero en versiones recientes el equipo migró ambas plataformas hacia V8 para unificar el motor y simplificar el mantenimiento (NativeScript Blog, 2024).

La documentación oficial del runtime de Android (NativeScript, s. f.-b) describe el mecanismo de la siguiente manera: "The Android Runtime may be thought of as 'The Bridge' between the JavaScript and Android worlds. A NativeScript application for Android is a standard native package (apk) which besides the JavaScript files embed the runtime as well" (párr. 1). Cuando el desarrollador escribe código como `android.widget.Toast.makeText()` en TypeScript, el runtime intercepta la llamada, busca la clase Java en los metadatos generados en tiempo de compilación y crea un proxy que representa el objeto nativo.

Metadatos y Binding Automático

Uno de los aspectos más innovadores de NativeScript es su sistema de generación automática de metadatos. Durante el proceso de construcción, las herramientas del framework inspeccionan los SDKs de iOS y Android y generan definiciones de tipos TypeScript para todas las clases y métodos disponibles. Según NativeScript GitHub (2023), "metadata for all of your public userland native APIs, and all SDK APIs, are automatically generated at build-time and bound at runtime" (párr. 1). El resultado es que el desarrollador dispone de autocompletado e inferencia de tipos para miles de APIs nativas directamente en su editor.

Este enfoque contrasta con React Native, donde el acceso a funcionalidades nativas no disponibles en el núcleo del framework requiere instalar módulos que sí necesitan código Java/Kotlin u Objective-C/Swift. En NativeScript, si una API existe en la plataforma, está disponible de forma inmediata desde JavaScript.

Renderizado de la Interfaz de Usuario

La interfaz de usuario en NativeScript se define mediante plantillas XML junto con estilos CSS. Sin embargo, a diferencia de un navegador web, NativeScript no interpreta el CSS para renderizar elementos HTML: utiliza las propiedades de estilo para configurar los atributos visuales de los componentes nativos correspondientes. Según ESIC (s. f.), los runtimes de NativeScript actúan como un puente que permite que el código JavaScript se comunique directamente con las APIs nativas, de modo que "el resultado es una aplicación que ofrece un rendimiento, una apariencia y una sensación indistinguibles de una app desarrollada con Swift/Objective-C (para iOS) o Kotlin/Java (para Android)" (párr. 3).

Lenguajes Utilizados

JavaScript

JavaScript es el lenguaje base sobre el que se construye NativeScript. Desarrolladores con experiencia en desarrollo web pueden comenzar a crear aplicaciones móviles nativas sin necesidad de aprender Kotlin, Swift, Java u Objective-C. El código JavaScript se ejecuta directamente sobre el runtime y pasa por procesos de optimización como tree-shaking y minificación mediante Webpack (NativeScript GitHub, 2023).

TypeScript

TypeScript es el lenguaje recomendado por el equipo de NativeScript para proyectos de mediana y gran escala. Al ser un superconjunto tipado de JavaScript, permite detectar errores en tiempo de desarrollo y mejora el autocompletado en editores como VS Code. El sistema de metadatos automáticos de NativeScript genera definiciones de tipos TypeScript para todas las APIs nativas, lo que convierte a TypeScript en la opción natural para aprovechar al máximo las capacidades del framework (NativeScript GitHub, 2023).

Frameworks de UI Compatibles

NativeScript admite múltiples frameworks de frontend para organizar la lógica y la interfaz de la aplicación. Loovatech (2024) señala que "NativeScript supports major frameworks (Angular, Vue, React, Svelte), Vanilla JS, or TS" (párr. 1), siendo Angular y el TypeScript puro los enfoques con mayor cantidad de ejemplos y soporte disponible. A continuación se detallan las principales opciones:

- Angular: framework más maduro dentro del ecosistema NativeScript, con soporte oficial desde la versión 2.0. Ofrece módulos, servicios, inyección de dependencias y RxJS.
- Vue.js: soportado desde 2017 a través del plugin NativeScript-Vue. Conveniente para equipos familiarizados con el ecosistema Vue.

- React: integración más reciente que permite utilizar componentes React para construir la interfaz nativa.
- Svelte y SolidJS: opciones de la comunidad orientadas a aplicaciones con alta eficiencia de renderizado.
- Vanilla JS/TypeScript: sin framework adicional, ideal para proyectos simples o para quienes prefieren control total sobre la arquitectura.

CSS para Estilos

NativeScript implementa un subconjunto de CSS para definir los estilos visuales. Aunque no soporta todas las propiedades del navegador, sí admite propiedades como color, font-size, margin, padding, border-radius, flexbox y animaciones básicas, lo que permite a los desarrolladores web aplicar su conocimiento de estilos de forma directa.

Casos Reales de Uso

Raiffeisen Bank

Uno de los casos empresariales más citados en el ecosistema de NativeScript es el de Raiffeisen Bank, uno de los grupos bancarios más grandes de Europa Central y Oriental. La institución utilizó NativeScript para construir aplicaciones móviles de gestión de datos de clientes, aprovechando la capacidad del framework de acceder a APIs de seguridad nativas como biometría y almacenamiento cifrado, características críticas en el sector financiero (Ideamotive, 2022).

SAP Mobile Development Kit

SAP, la empresa de software empresarial más grande del mundo en el segmento ERP, eligió NativeScript como tecnología base de su Mobile Development Kit (MDK). Ideamotive

(2022) señala que "SAP, the world's largest ERP company, is also the one that uses NativeScript. The company has decided to use this framework to build its Mobile Development Kit" (párr. 8). Esta adopción por parte de una empresa de esa escala valida la capacidad del framework para aplicaciones de nivel empresarial.

Daily Nanny

Daily Nanny es una aplicación de gestión de servicios de cuidado infantil que utiliza NativeScript para ofrecer una experiencia consistente en iOS y Android desde una única base de código. La aplicación aprovecha las capacidades de notificaciones push nativas y la integración con calendarios del dispositivo para coordinar horarios entre padres y cuidadores (Brainhub, 2026).

Sector de Atención al Cliente

Empresas del sector de servicio al cliente han adoptado NativeScript para portar aplicaciones Angular existentes hacia plataformas móviles nativas, reduciendo el tiempo y costo de desarrollo al reutilizar la lógica de negocio y los servicios ya implementados. StackShare (s. f.) documenta más de 500 stacks tecnológicos que reportan el uso de NativeScript, incluyendo organizaciones en los sectores fintech y consultoría de software.

Comparación Técnica con Android Nativo

El desarrollo nativo en Android implica utilizar Kotlin o Java con el SDK oficial, compilando el código a bytecode que la máquina virtual ART ejecuta directamente en el dispositivo. NativeScript, en cambio, ejecuta código JavaScript sobre el motor V8 y traduce las llamadas a APIs nativas en tiempo de ejecución. La siguiente tabla resume los puntos de comparación más relevantes:

Criterio	NativeScript	Android Nativo
Lenguaje principal	JavaScript / TypeScript	Kotlin / Java
Renderizado UI	Componentes nativos reales	Componentes nativos reales
Acceso a APIs	100% directo vía runtime JS	100% directo nativo
Multiplataforma	iOS, Android, visionOS	Solo Android
Curva de aprendizaje	Media (JS/TS conocido)	Alta (Kotlin/Java)
Rendimiento	Cercano al nativo	Óptimo nativo
Tamaño de comunidad	Pequeña pero activa	Muy grande
Costo de desarrollo	Menor (base de código única)	Mayor (por plataforma)
Tamaño del APK	Levemente mayor	Menor
Licencia	Apache 2.0 (código abierto)	Apache 2.0

Rendimiento

El desarrollo nativo ofrece el mejor rendimiento posible, ya que el código Kotlin/Java se compila directamente para la plataforma objetivo. NativeScript presenta un rendimiento cercano al nativo para la mayoría de las aplicaciones de uso empresarial. GeeksforGeeks (2025) indica que NativeScript "delivers near-native performance with direct compilation to native code" y que es "ideal for apps needing smooth UX and tight integration with device features" (párr. 6). La diferencia de rendimiento se vuelve perceptible únicamente en aplicaciones con cálculos intensivos o gráficos muy exigentes.

Mantenimiento del Código

Una de las ventajas más tangibles de NativeScript sobre el desarrollo nativo puro es la reducción del código a mantener. Una organización que necesite aplicaciones en iOS y Android con desarrollo nativo debe mantener dos bases de código completamente independientes. Con NativeScript, según la documentación oficial, es posible compartir aproximadamente el 90% del código entre ambas plataformas, reduciendo costos operativos y de desarrollo (NativeScript, s. f.-a).

Acceso a Funcionalidades del Dispositivo

Tanto el desarrollo nativo como NativeScript ofrecen acceso completo a todas las funcionalidades del dispositivo: cámara, GPS, sensores, almacenamiento, notificaciones, Bluetooth y NFC, entre otros. La diferencia radica en el mecanismo de acceso: en Android nativo el desarrollador interactúa directamente con el SDK en Kotlin/Java, mientras que en NativeScript lo hace desde JavaScript a través del runtime, sin necesidad de escribir código nativo adicional (NativeScript, s. f.-b).

Ventajas y Desventajas

Con base en la investigación realizada y el proceso de familiarización con el framework, el grupo identificó las siguientes ventajas y desventajas:

Ventajas	Desventajas
Acceso 100% directo a APIs nativas sin código adicional	Comunidad reducida comparada con React Native o Flutter
Un único código para iOS y Android	No soporta HTML ni DOM estándar del navegador
Compatible con Angular, Vue, React y Svelte	Depuración requiere emulador o dispositivo físico
UI con componentes nativos reales (sin WebView)	APK/IPA levemente mayor en tamaño
Soporte completo para TypeScript	Requiere conocer APIs de iOS/Android para casos avanzados
Open source bajo licencia Apache 2.0	Documentación menos extensa que competidores
Compatible con paquetes npm y CocoaPods	Algunos componentes UI avanzados son de pago
Rendimiento cercano al desarrollo nativo	Menor demanda en el mercado laboral actual

Ecosistema de Herramientas y Soporte de Desarrollo

NativeScript CLI

La interfaz de línea de comandos de NativeScript (@nativescript/cli) es el punto de entrada principal para crear, compilar y desplegar proyectos. Mediante comandos como ns create, ns run android y ns build ios, el desarrollador gestiona todo el ciclo de vida de la

aplicación sin necesidad de abrir Android Studio o Xcode para las tareas habituales. El CLI incluye además ns doctor, un diagnóstico automatizado que verifica que el entorno de desarrollo esté correctamente configurado, detecta versiones incompatibles del SDK de Android y reporta dependencias faltantes (NativeScript, s. f.-a).

Marketplace de Plugins y Ecosistema npm

El ecosistema de plugins de NativeScript se distribuye a través del registro oficial de npm bajo el espacio de nombres @nativescript/. Existen plugins mantenidos oficialmente para funcionalidades críticas como cámara (@nativescript/camera), geolocalización (@nativescript/geolocation), notificaciones push (@nativescript/firebase), almacenamiento local cifrado (@nativescript/secure-storage) y biometría (@nativescript/biometrics). Esta cobertura de funcionalidades nativas mediante plugins de primera parte reduce la dependencia de soluciones de terceros no mantenidas, un problema recurrente en el ecosistema de React Native (NativeScript GitHub, 2023).

Integración con Visual Studio Code

El equipo de NativeScript mantiene una extensión oficial para Visual Studio Code que proporciona depuración en tiempo real sobre emulador o dispositivo físico, con puntos de quiebre en el código TypeScript original (no en el JavaScript compilado), inspección de variables y evaluación de expresiones. La extensión se integra con las Chrome DevTools para permitir la depuración del hilo de JavaScript, lo que resulta familiar para desarrolladores web. Esta integración con el editor más utilizado en el ecosistema TypeScript reduce la curva de aprendizaje operacional del framework y agiliza el ciclo de desarrollo (ESIC, s. f.).

Retos Encontrados

Configuración del Entorno de Desarrollo

El primer reto al trabajar con NativeScript es la configuración del entorno. A diferencia de un proyecto web convencional, NativeScript requiere tener instalados Node.js, el CLI del framework (@nativescript/cli), Android Studio con el SDK correspondiente y, en el caso de iOS, Xcode en un sistema macOS. Este proceso implica la configuración de variables de entorno y puede presentar conflictos de versiones entre dependencias.

Comunidad Reducida

En comparación con React Native o Flutter, NativeScript cuenta con una comunidad considerablemente más pequeña. Loovatech (2024) señala que "the active users of the NativeScript server on Discord can be counted on one's fingers" (párr. 3) y que "NativeScript's market share fell from 11% to 3% from 2019 to 2022" (párr. 4). Esto se traduce en menos respuestas disponibles en Stack Overflow y menos tutoriales en español, lo que aumenta el tiempo de resolución de problemas técnicos.

Compatibilidad con Librerías npm

Si bien NativeScript es compatible con una gran cantidad de paquetes npm, no todos los paquetes diseñados para entornos web o Node.js funcionan en NativeScript, dado que el framework no implementa el DOM ni el entorno de navegador estándar. Esto requiere una evaluación caso por caso de las dependencias que el proyecto necesite utilizar.

Depuración en Dispositivo o Emulador

La depuración de aplicaciones NativeScript requiere un emulador configurado o un dispositivo físico conectado. Aunque NativeScript ofrece integración con las Chrome

DevTools para depuración, el flujo de trabajo es menos inmediato que en el desarrollo web. Según NativeScript GitHub (2023), el proceso de inspección requiere ejecutar un comando específico que genera una URL para abrir en Chrome o Chromium (párr. 12).

Análisis Crítico del Grupo

Tras el proceso de investigación, el grupo considera que NativeScript representa una opción técnicamente sólida para el desarrollo de aplicaciones empresariales multiplataforma, especialmente en equipos con experiencia previa en TypeScript o Angular. Su propuesta de valor más diferenciadora —el acceso 100% directo a las APIs nativas desde JavaScript sin necesidad de código nativo adicional— es genuinamente única en el ecosistema y resuelve un problema real que existe en frameworks competidores.

Sin embargo, el grupo también reconoce que la menor popularidad del framework en comparación con Flutter o React Native representa un desafío práctico. Pagepro (2024) señala que "as of 2024, the general trend suggests that React Native continues to lead" (párr. 9) en términos de adopción del mercado. La escasez de recursos educativos en español y el menor número de ofertas laborales que mencionan NativeScript son factores que deben considerarse tanto para este proyecto académico como para decisiones de largo plazo en el ámbito profesional.

En el contexto específico de la problemática asignada —un sistema de gestión de órdenes y entregas— NativeScript resulta una elección pertinente. La aplicación requiere notificaciones push para actualizar el estado de los pedidos, almacenamiento local para funcionamiento parcial sin conexión, integración con cámara para evidencia fotográfica y acceso a servicios de localización; todas capacidades que NativeScript puede implementar directamente sin depender de plugins de terceros ni código nativo adicional.

El principal aprendizaje de esta investigación no es únicamente conocer NativeScript, sino comprender el espectro completo de aproximaciones al desarrollo móvil —desde el nativo puro hasta el híbrido basado en WebView— y ser capaz de justificar técnicamente la elección de una tecnología en función de los requerimientos concretos de un proyecto real.

Desde la perspectiva del stack tecnológico adoptado por el grupo —ASP.NET Core 8 en C# como backend, autenticación con JWT Bearer y base de datos SQL Server—, NativeScript demuestra una compatibilidad natural. El cliente móvil consume la API REST mediante el módulo nativo de red de NativeScript, que admite cabeceras HTTP personalizadas para enviar el token JWT en cada solicitud protegida. El patrón MVVM implementado con TypeScript en el frontend móvil facilita la separación entre la lógica de negocio y la presentación, lo cual es consistente con las buenas prácticas de arquitectura que el enunciado del proyecto requiere. Esta alineación entre las capacidades del framework y las decisiones de arquitectura adoptadas confirma que NativeScript fue una elección técnicamente apropiada para la problemática de gestión de órdenes y entregas asignada al grupo.

Referencias

Brainhub. (2026, 5 de marzo). NativeScript vs React Native – Which one to choose.

<https://brainhub.eu/library/javascript-mobile-development-nativescript-react-native>

ESIC Business & Marketing School. (s. f.). NativeScript: qué es, características y ventajas.

<https://www.esic.edu/rethink/tecnologia/nativescript-que-es-y-caracteristicas-c>

GeeksforGeeks. (2025, 5 de agosto). NativeScript vs React Native: Which one to choose in

2025. <https://www.geeksforgeeks.org/blogs/react-native-vs-nativescript/>

Ideamotive. (2022, 2 de febrero). NativeScript vs React Native for mobile app development.

<https://www.ideamotive.co/blog/nativescript-vs-react-native>

InfoQ. (2020, 8 de diciembre). NativeScript now a member of the OpenJS Foundation.

<https://www.infoq.com/news/2020/12/nativescript-joins-openjs/>

Loovatech. (2024, 15 de enero). NativeScript in a cross-platform development world. Medium.

<https://medium.com/@loovatech/nativescript-in-a-cross-platform-development-world-f052b3287a9b>

NativeScript. (2014, 12 de junio). Announcing NativeScript – cross-platform framework for

building native mobile applications. Telerik Blog.

<https://www.telerik.com/blogs/announcing-nativescript---cross-platform-framework-for-building-native-mobile-applications>

NativeScript. (2020, 1 de junio). The next chapter for NativeScript: nStudio. NativeScript Blog.

<https://blog.nativescript.org/the-next-chapter-for-nativescript-nstudio/>

NativeScript. (s. f.-a). iOS Runtime overview. NativeScript Documentation.

<https://old.docs.nativescript.org/core-concepts/ios-runtime/overview>

NativeScript. (s. f.-b). Android Runtime overview. NativeScript Documentation.

<https://old.docs.nativescript.org/core-concepts/android-runtime/overview>

NativeScript GitHub. (2023, 25 de noviembre). Deep dive: How NativeScript's JS native bindings work. GitHub Wiki. <https://github.com/NativeScript/NativeScript/wiki/Deep-dive:-How-NativeScript's-JS--native-bindings-work>

NativeScript Blog. (2024). The new iOS runtime, powered by V8. <https://blog.nativescript.org/the-new-ios-runtime-powered-by-v8/>

OpenJS Foundation. (2020, 7 de diciembre). NativeScript joins OpenJS Foundation as incubating project. <https://openjsf.org/blog/2020/12/07/nativescript-joins-openjs-foundation-as-incubating-project/>

Pagepro. (2024, 9 de septiembre). React Native vs NativeScript: Comparison. <https://pagepro.co/blog/react-native-nativescript-comparison/>

StackShare. (s. f.). NativeScript – reviews, pros & cons, companies using NativeScript. <https://stackshare.io/nativescript>